

Week-6: (DAY-26) Hands-On – Keerthivasan R

Problem: 1- SRP – Single Responsibility Principle

Scenario: Student Report Generator

A training institute like Codempower Academy maintains student information and generates performance reports. Currently, one class performs student data storage and report generation, which makes the code difficult to maintain.

Requirements:

1. Create a **Student class** with properties:
 - StudentId
 - StudentName
 - Marks
2. Create a class responsible for **managing student data**.
3. Create a separate class responsible for **generating reports**.
4. The report should display:

Security Requirements (Secure Coding Practices)

Students must implement the following security measures:

Technical Constraints:

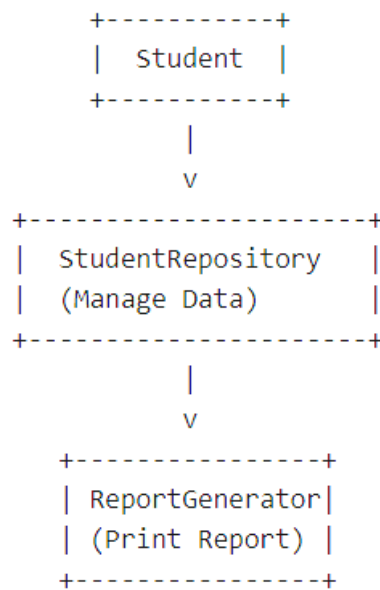
- Use **C# (.NET Console Application)**.
- Each class must have **only one responsibility**.
- Do not mix **data storage and report generation** logic in the same class.

Expectations:

Students should implement at least **three classes**:

- Student
- StudentRepository
- ReportGenerator

Program Flow Diagram



Learning Outcome:

Students will learn:

- Meaning of **Single Responsibility Principle**
- How to **separate responsibilities**
- How SRP improves **maintainability and testing**

CODE:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace HandsOnDAY26
{
    class Student // holds data
    {
        public int Id { get; set; }
        public string Name { get; set; }
        public int Marks { get; set; }

        public Student(int id, string name, int marks )
        {
            Id = id;
            Name = name;
            Marks = marks;
        }
    }
}
```

```

}

class StudentRepository // manages data
{
    private List<Student> students = new List<Student>();

    public void AddStudent(Student student)
    {
        students.Add(student);
    }

    public List<Student> GetAllStudents()
    {
        return students;
    }
}

class ReportGenerator // generates report
{
    public void GenerateReport(List<Student> students)
    {
        Console.WriteLine("----- Student Report -----");

        foreach (var student in students)
        {
            string grade = GetGrade(student.Marks);

            Console.WriteLine($"ID: {student.Id}");
            Console.WriteLine($"Name: {student.Name}");
            Console.WriteLine($"Marks: {student.Marks}");
            Console.WriteLine($"Grade: {grade}");
            Console.WriteLine("-----");
        }
    }

    private string GetGrade(int marks)
    {
        if (marks >= 90)
        {
            return "A";
        }
        else if (marks >= 75)
        {
            return "B";
        }
        else if (marks >= 50)
        {
            return "C";
        }
        else
        {
            return "Fail";
        }
    }
}

internal class Problem_01_SRP // SRP - Single Responsibility Principle
{
    static void Main(string[] args)
    {
        StudentRepository repo = new StudentRepository();

        //adding students
        repo.AddStudent(new Student(1, "Arun", 92));
        repo.AddStudent(new Student(2, "Divya", 76));
        repo.AddStudent(new Student(3, "Kiran", 45));
    }
}

```

```

        //Generate Report
        ReportGenerator report = new ReportGenerator();
        report.GenerateReport(repo.GetAllStudents());
    }
}

```

OUTPUT:

```

----- Student Report -----
ID: 1
Name: Arun
Marks: 92
Grade: A
-----
ID: 2
Name: Divya
Marks: 76
Grade: B
-----
ID: 3
Name: Kiran
Marks: 45
Grade: Fail
-----

C:\Users\keerthi\OneDrive\Desktop\upGrad\DAY-26\HandsOnDAY26\HandsOnDAY26\bin\Debug\net8.0\HandsOnDAY26.
exe (process 25776) exited with code 0 (0x0).
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically
close the console when debugging stops.
Press any key to close this window . . .|

```

Problem: 2- OCP – Open Closed Principle

Scenario: Discount Calculation System

An e-commerce system calculates discounts for different customer types:

- Regular Customer
- Premium Customer
- VIP Customer

The system should allow **adding new discount types without modifying existing code**.

Requirements:

1. Create an abstract class or interface:
IDiscountStrategy
2. Implement discount classes:
 - RegularCustomerDiscount
 - PremiumCustomerDiscount
 - VipCustomerDiscount
3. Each class should implement a method:
CalculateDiscount(double amount)

Technical Constraints:

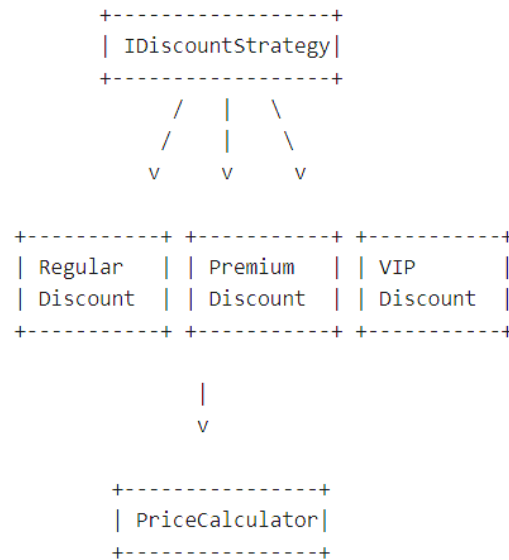
- Use interface or abstract class
- Existing classes should not be modified when adding new discounts
- Follow Open for Extension, Closed for Modification

Expectations:

Students should implement:

- IDiscountStrategy
- 3 Discount Classes
- A class that calculates the final price

Program Flow Diagram:



Learning Outcome:

Students will understand:

- **Extending functionality without modifying existing code**
- Use of **interfaces and polymorphism**
- Real-world use of **Strategy Pattern**

CODE:

```

using System;
using System.Collections.Generic;
namespace HandsOnDAY26
{
    // 1. Interface
    public interface IDiscountStrategy
    {
        double CalculateDiscount(double amount);
    }

    // 2. Discount Classes
    public class RegularCustomerDiscount : IDiscountStrategy
    {
        public double CalculateDiscount(double amount)
        {
            return amount * 0.05; // 5% discount
        }
    }

    public class PremiumCustomerDiscount : IDiscountStrategy
    {
        public double CalculateDiscount(double amount)
        {
            return amount * 0.10; // 10% discount
        }
    }

    public class VipCustomerDiscount : IDiscountStrategy
    {
        public double CalculateDiscount(double amount)
        {

```

```

        return amount * 0.20; // 20% discount
    }
}
// 3. Price Calculator (uses strategy)
public class PriceCalculator
{
    private IDiscountStrategy _discountStrategy;

    public PriceCalculator(IDiscountStrategy discountStrategy)
    {
        _discountStrategy = discountStrategy;
    }

    public double GetFinalPrice(double amount)
    {
        double discount = _discountStrategy.CalculateDiscount(amount);
        return amount - discount;
    }
}
// Main Class
internal class Problem_02_OCP {
    static void Main(string[] args) {
        double amount = 1000;

        // Regular Customer
        PriceCalculator regular = new PriceCalculator(new RegularCustomerDiscount());
        Console.WriteLine("Regular Customer Final Price: " + regular.GetFinalPrice(amount));

        // Premium Customer
        PriceCalculator premium = new PriceCalculator(new PremiumCustomerDiscount());
        Console.WriteLine("Premium Customer Final Price: " + premium.GetFinalPrice(amount));

        // VIP Customer
        PriceCalculator vip = new PriceCalculator(new VipCustomerDiscount());
        Console.WriteLine("VIP Customer Final Price: " + vip.GetFinalPrice(amount));
    }
}

```

OUTPUT:

```

Regular Customer Final Price: 950
Premium Customer Final Price: 900
VIP Customer Final Price: 800

C:\Users\keerthi\OneDrive\Desktop\upGrad\DAY-26\HandsOnDAY26\HandsOnDAY26\bin\
Debug\net8.0\HandsOnDAY26.exe (process 22176) exited with code 0 (0x0).
To automatically close the console when debugging stops, enable Tools->Options
->Debugging->Automatically close the console when debugging stops.
Press any key to close this window . . .|

```

Problem 3- LSP – Liskov Substitution Principle

Scenario: Shape Area Calculator

A graphics application calculates the **area of different shapes**.

Any derived class should be able to **replace the base class without breaking functionality**.

Requirements:

1. Create a base class or interface:
 - Shape
2. Derived classes:
 - Rectangle
 - Circle
3. Each shape should implement:
 - CalculateArea()
4. A method should accept **Shape object** and calculate area.

Technical Constraints:

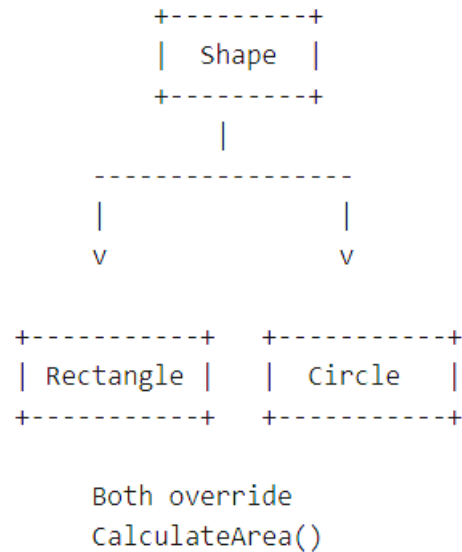
- Use method overriding
- Derived classes must not break base class behavior

Expectations:

Students should demonstrate that the program works correctly when:

- Rectangle object is used
- Circle object is used

Program Flow Diagram:



Learning Outcome:

Students will learn:

- Meaning of **substitutability**
- Importance of **correct inheritance**
- How polymorphism supports LSP

CODE:

```
using System;
namespace HandsOnDAY26{
    // 1. Base Class
    public abstract class Shape {
        public abstract double CalculateArea();
    }

    // 2. Rectangle Class
    public class Rectangle : Shape {
        public double Length { get; set; }
        public double Width { get; set; }

        public Rectangle(double length, double width) {
            Length = length;
            Width = width;
        }

        public override double CalculateArea() {
            return Length * Width;
        }
    }

    // 3. Circle Class
    public class Circle : Shape
```

```

{
    public double Radius { get; set; }

    public Circle(double radius)
    {
        Radius = radius;
    }

    public override double CalculateArea()
    {
        return Math.PI * Radius * Radius;
    }
}

// Main Class
internal class Problem_03_LSP
{
    // Method accepting Shape (LSP in action)
    public static void PrintArea(Shape shape)
    {
        Console.WriteLine("Area: " + shape.CalculateArea());
    }

    static void Main(string[] args)
    {
        // Rectangle object
        Shape rectangle = new Rectangle(10, 5);
        PrintArea(rectangle);

        // Circle object
        Shape circle = new Circle(7);
        PrintArea(circle);
    }
}
}

```

OUTPUT:

```

Area: 50
Area: 153.93804002589985

C:\Users\keerthi\OneDrive\Desktop\upGrad\DAY-26\HandsOnDAY26\HandsOnDAY26\bin\
Debug\net8.0\HandsOnDAY26.exe (process 26672) exited with code 0 (0x0).
To automatically close the console when debugging stops, enable Tools->Options
->Debugging->Automatically close the console when debugging stops.
Press any key to close this window . . .|

```

Problem 4- ISP – Interface Segregation Principle

Scenario: Office Printer System

An office has different machines:

- Basic Printer (Print only)
- Advanced Printer (Print + Scan + Fax)

If we create a **single large interface**, basic printers will be forced to implement unnecessary methods.

The task is to **split the interface into smaller interfaces**.

Requirements:

Create the following interfaces:

- IPrinter
- IScanner
- IFax

Classes:

- BasicPrinter → Print only
- AdvancedPrinter → Print + Scan + Fax

Technical Constraints:

- Follow **Interface Segregation Principle**
- Classes should **not implement unnecessary methods**

Expectations:

Students should implement:

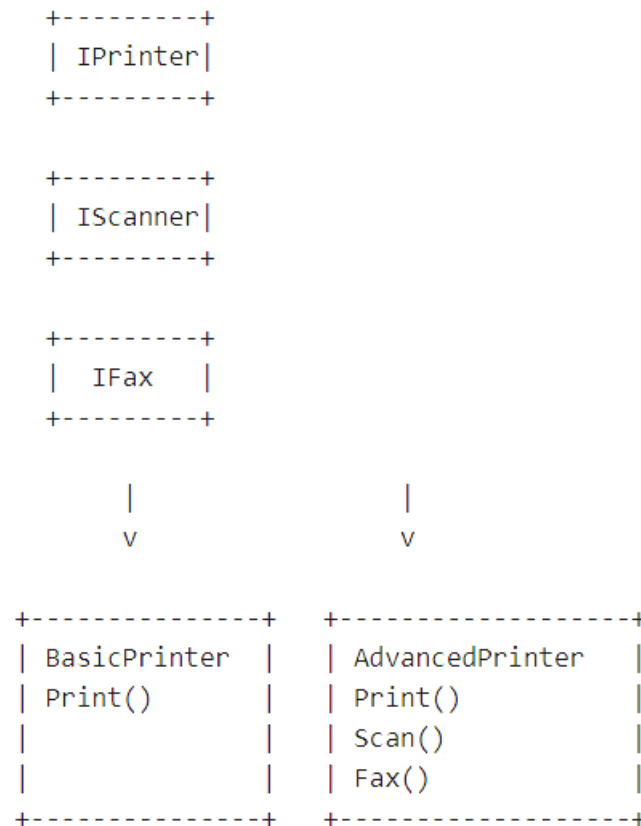
Interfaces

- IPrinter
- IScanner
- IFax

Classes

- BasicPrinter
- AdvancedPrinter

Program Flow Diagram:



Learning Outcome:

Students will learn:

- Why **large interfaces are bad**
- How to design **small focused interfaces**
- Better **maintainability and flexibility**

CODE:

```
using System;

namespace HandsOnDAY26
{
    // 1. Small Interfaces

    public interface IPrinter
    {
        void Print();
    }

    public interface IScanner
```

```

{
    void Scan();
}

public interface IFax
{
    void Fax();
}

// 2. Basic Printer (only Print)
public class BasicPrinter : IPrinter
{
    public void Print()
    {
        Console.WriteLine("Basic Printer: Printing document...");
    }
}

// 3. Advanced Printer (Print + Scan + Fax)
public class AdvancedPrinter : IPrinter, IScanner, IFax
{
    public void Print()
    {
        Console.WriteLine("Advanced Printer: Printing document...");
    }

    public void Scan()
    {
        Console.WriteLine("Advanced Printer: Scanning document...");
    }

    public void Fax()
    {
        Console.WriteLine("Advanced Printer: Sending fax...");
    }
}

// Main Class
internal class Problem_04_ISP
{
    static void Main(string[] args)
    {
        // Basic Printer
        IPrinter basic = new BasicPrinter();
        basic.Print();

        Console.WriteLine();

        // Advanced Printer
        AdvancedPrinter advanced = new AdvancedPrinter();
        advanced.Print();
        advanced.Scan();
        advanced.Fax();
    }
}

```

OUTPUT:

```
Basic Printer: Printing document...

Advanced Printer: Printing document...
Advanced Printer: Scanning document...
Advanced Printer: Sending fax...

C:\Users\keerthi\OneDrive\Desktop\upGrad\DAY-26\HandsOnDAY26\HandsOnDAY26\bin\Debug\net8.0\HandsOnDAY26.exe (process 17680) exited with code 0 (0x0).
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the console when debugging stops.
Press any key to close this window . . .|
```

Problem 5- Implementing Singleton Pattern

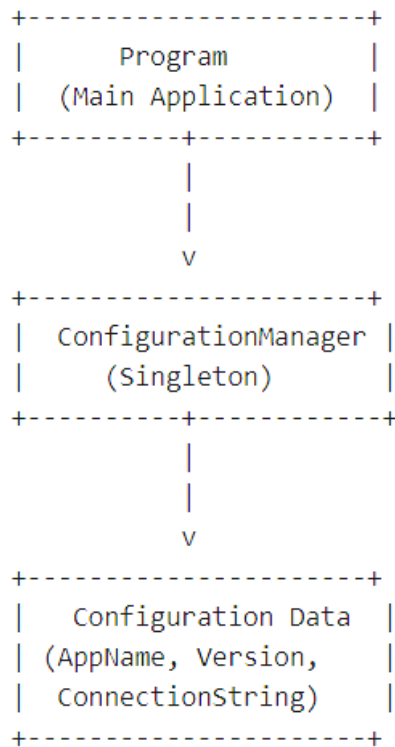
Scenario: Application Configuration Manager

A company is developing a **console-based inventory management system**. The application needs to read **application settings (e.g., database connection string, app name, version)** from a configuration manager.

Since configuration settings should be **loaded only once and shared across the entire application**, multiple instances of the configuration manager must be prevented.

Therefore, the development team decides to implement the **Singleton Design Pattern**.

Program Flow Diagram:



Requirements:

Students must implement:

1. A class named **ConfigurationManager**
2. Ensure **only one instance** of the class can be created.
3. Provide a method:

GetInstance()

to retrieve the single object.

4. Store configuration values such as:
 - ApplicationName
 - Version
 - DatabaseConnectionString
5. Demonstrate that multiple calls to GetInstance() return **the same instance**.

Technical Constraints:

- Use **private constructor**.
- Use **static instance variable**.

- Use **thread-safe implementation (basic level optional)**.
- Console application using **C# (.NET)**.

Expectations:

Students should:

- Prevent object creation using new.
- Access instance using:
ConfigurationManager.GetInstance()
- Print configuration details from multiple method calls.

Learning Outcome:

After completing this problem, learners will understand:

- Why Singleton is used
- How to restrict object creation
- Global shared objects
- Basic design pattern implementation in C#

CODE:

```
using System;
namespace HandsOnDAY26{
    // 1. Singleton Class
    public class ConfigurationManager {
        // Static instance (only one object)
        private static ConfigurationManager instance;
        // Lock object for thread safety (optional basic level)
        private static readonly object lockObj = new object();
        // Properties
        public string ApplicationName { get; set; }
        public string Version { get; set; }
        public string DatabaseConnectionString { get; set; }
        // Private constructor (prevents new keyword)
        private ConfigurationManager() {
            // Default values
            ApplicationName = "Inventory System";
            Version = "1.0.0";
            DatabaseConnectionString = "Server=localhost;Database=InventoryDB;";
        }

        // GetInstance method
        public static ConfigurationManager GetInstance() {
            if (instance == null) {
                lock (lockObj) {
                    if (instance == null) {
                        instance = new ConfigurationManager();
                    }
                }
            }
        }
    }
}
```



```

    }
    }
    return instance;
}
}
// Main Class
internal class Problem_05_Singleton_Implementation {
    static void Main(string[] args) {
        // First call
        ConfigurationManager config1 = ConfigurationManager.GetInstance();
        // Second call
        ConfigurationManager config2 = ConfigurationManager.GetInstance();
        // Print details from both instances
        Console.WriteLine("First Instance:");
        Console.WriteLine(config1.ApplicationName);
        Console.WriteLine(config1.Version);
        Console.WriteLine(config1.DatabaseConnectionString);
        Console.WriteLine();
        Console.WriteLine("Second Instance:");
        Console.WriteLine(config2.ApplicationName);
        Console.WriteLine(config2.Version);
        Console.WriteLine(config2.DatabaseConnectionString);
        Console.WriteLine();
        // Check if both instances are same
        Console.WriteLine("Are both instances same? " + (config1 == config2));
    }
}

```

OUTPUT:

```

First Instance:
Inventory System
1.0.0
Server=localhost;Database=InventoryDB;

Second Instance:
Inventory System
1.0.0
Server=localhost;Database=InventoryDB;

Are both instances same? True

C:\Users\keerthi\OneDrive\Desktop\upGrad\DAY-26\HandsOnDAY26\HandsOnDAY26\bin\Debug\net8.0\HandsOnDAY26.exe (process 27168) exited with code 0 (0x0).
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the console when debugging stops.
Press any key to close this window . . .|

```

Problem 6- Implementing Factory Pattern

Scenario: Notification Service

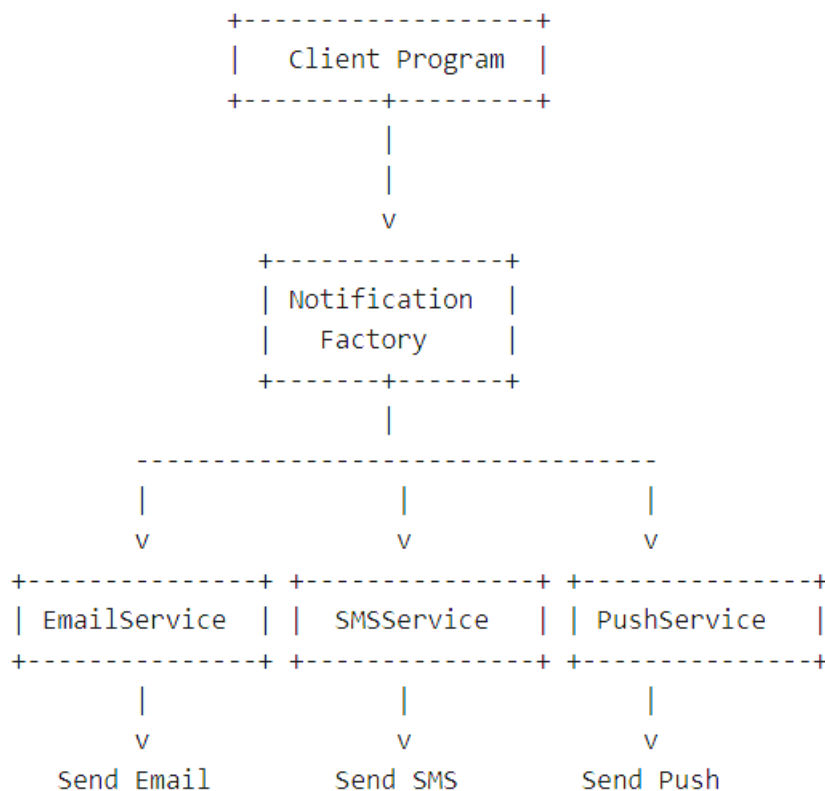
A company application sends **notifications to users** using different communication channels:

- Email
- SMS
- Push Notification

The application should **not directly create objects using new**.

Instead, a **Factory class** should determine which notification service to create.

Program Flow:



Requirements:

Students must implement:

- **Interface**

INotification

- **Method:**

Send(string message)

Classes

- EmailNotification
- SMSNotification
- PushNotification

Each class should implement **INotification**.

Factory Class

NotificationFactory

Method:

CreateNotification(string type)

Example:

CreateNotification("email")

CreateNotification("sms")

CreateNotification("push")

Technical Constraints

- Use **interface-based design**.
- Client should **not instantiate concrete classes directly**.
- Use **Factory Pattern** to create objects.
- Language: **C# Console Application**.

Expectations:

Students should demonstrate:

```
NotificationFactory factory = new NotificationFactory();
```

```
var notification = factory.CreateNotification("email");
```

```
notification.Send("Welcome to our service!");
```

Learning Outcome:

Students will learn:

- Decoupling object creation from usage
- Interface-based programming
- Open/Closed Principle basics
- Real-world object creation management

CODE:

```
using System;

namespace HandsOnDAY26
{
    // 1. Interface
    public interface INotification
    {
        void Send(string message);
    }

    // 2. Concrete Classes

    public class EmailNotification : INotification
    {
        public void Send(string message)
        {
            Console.WriteLine("Email Sent: " + message);
        }
    }

    public class SMSNotification : INotification
    {
        public void Send(string message)
        {
            Console.WriteLine("SMS Sent: " + message);
        }
    }

    public class PushNotification : INotification
    {
        public void Send(string message)
        {
            Console.WriteLine("Push Notification Sent: " + message);
        }
    }

    // 3. Factory Class
    public class NotificationFactory
    {
        public INotification CreateNotification(string type)
        {
            switch (type.ToLower())
            {
                case "email":
                    return new EmailNotification();

                case "sms":
                    return new SMSNotification();

                case "push":
```

```

        return new PushNotification();
    }
    default:
        throw new ArgumentException("Invalid notification type");
    }
}

// Main Class
internal class Problem_06_Implementing_Factory_Pattern
{
    static void Main(string[] args)
    {
        NotificationFactory factory = new NotificationFactory();

        var email = factory.CreateNotification("email");
        email.Send("Welcome to our service!");

        var sms = factory.CreateNotification("sms");
        sms.Send("Your OTP is 123456");

        var push = factory.CreateNotification("push");
        push.Send("You have a new alert!");
    }
}

```

OUTPUT:

```

Email Sent: Welcome to our service!
SMS Sent: Your OTP is 123456
Push Notification Sent: You have a new alert!

C:\Users\keerthi\OneDrive\Desktop\upGrad\DAY-26\HandsOnDAY26\HandsOnDAY26\bin\Debug\net8.0\HandsOnDAY26.exe (process 4348) exited with code 0 (0x0).
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the console when debugging stops.
Press any key to close this window . . .|

```

Problem 7- Implementing Repository Pattern

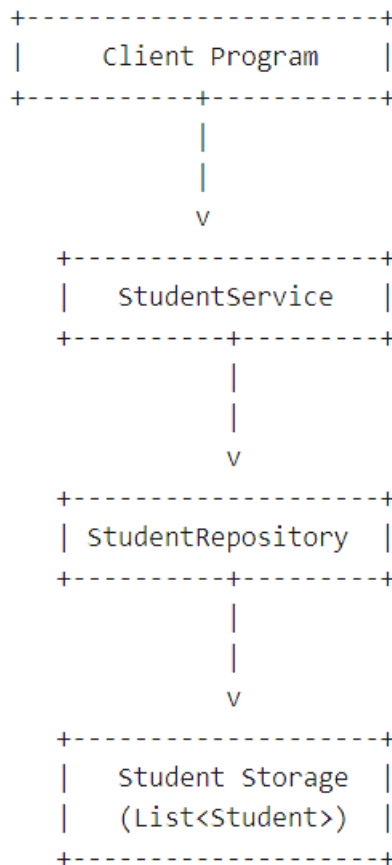
Scenario: Student Data Management System

A training institute needs a system to manage **student information**.

Instead of directly writing database access code in the main program, the development team decides to use the **Repository Pattern**.

The repository will act as a **data access layer** between the application and the data source.

For simplicity, student data will be stored in a **List collection**.



Requirements:

Create the following components:

Entity Class:

Student

Properties:

StudentId
StudentName
Course

Repository Interface:

IStudentRepository

Methods:

AddStudent(Student student)

GetAllStudents()

GetStudentById(int id)

DeleteStudent(int id)

Repository Implementation:

StudentRepository

Store data using:

List<Student>

Main Program

Demonstrate:

- Adding students
- Viewing students
- Finding student by ID
- Deleting student

Expectations:

Students should implement separation between:

Business Logic



Repository Layer



Data Storage

Learning Outcome:

Students will understand:

- Separation of concerns
- Data access abstraction
- Clean architecture basics
- How repositories simplify data operations

CODE:

```
using System;
using System.Collections.Generic;
using System.Linq;

namespace HandsOnDAY26
{
    // Entity Class
    public class RepoStudent
    {
        public int StudentId { get; set; }
        public string StudentName { get; set; }
        public string Course { get; set; }

        public RepoStudent(int id, string name, string course)
        {
            StudentId = id;
            StudentName = name;
            Course = course;
        }
    }

    // Repository Interface
    public interface IRepoStudentRepository
    {
        void AddStudent(RepoStudent student);
        List<RepoStudent> GetAllStudents();
        RepoStudent GetStudentById(int id);
        void DeleteStudent(int id);
    }

    // Repository Implementation
    public class RepoStudentRepository : IRepoStudentRepository
    {
        private List<RepoStudent> students = new List<RepoStudent>();

        public void AddStudent(RepoStudent student)
        {
            students.Add(student);
        }

        public List<RepoStudent> GetAllStudents()
        {
            return students;
        }
    }
}
```



```

public RepoStudent GetStudentById(int id)
{
    return students.FirstOrDefault(s => s.StudentId == id);
}

public void DeleteStudent(int id)
{
    var student = GetStudentById(id);
    if (student != null)
    {
        students.Remove(student);
    }
}
}

// Main Class
internal class Implementing_Repository_Pattern
{
    static void Main(string[] args)
    {
        // ☒ Correct: Use implementation class
        IRepoStudentRepository repo = new RepoStudentRepository();

        // Adding Students
        repo.AddStudent(new RepoStudent(1, "Arun", "Java"));
        repo.AddStudent(new RepoStudent(2, "Divya", "Python"));
        repo.AddStudent(new RepoStudent(3, "Kiran", "C#"));

        // View All Students
        Console.WriteLine("All Students:");
        foreach (var s in repo.GetAllStudents())
        {
            Console.WriteLine($"ID: {s.StudentId}, Name: {s.StudentName}, Course:
{s.Course}");
        }

        Console.WriteLine();

        // Find Student by ID
        Console.WriteLine("Search Student with ID 2:");
        var student = repo.GetStudentById(2);
        if (student != null)
        {
            Console.WriteLine($"Found: {student.StudentName}, Course: {student.Course}");
        }
        else
        {
            Console.WriteLine("Student not found");
        }

        Console.WriteLine();

        // Delete Student
        Console.WriteLine("Deleting Student with ID 1...");
        repo.DeleteStudent(1);

        Console.WriteLine("Students After Deletion:");
        foreach (var s in repo.GetAllStudents())
        {
            Console.WriteLine($"ID: {s.StudentId}, Name: {s.StudentName}, Course:
{s.Course}");
        }
    }
}

```

OUTPUT:

```
All Students:
ID: 1, Name: Arun, Course: Java
ID: 2, Name: Divya, Course: Python
ID: 3, Name: Kiran, Course: C#

Search Student with ID 2:
Found: Divya, Course: Python

Deleting Student with ID 1...
Students After Deletion:
ID: 2, Name: Divya, Course: Python
ID: 3, Name: Kiran, Course: C#

C:\Users\keerthi\OneDrive\Desktop\upGrad\DAY-26\HandsOnDAY26\HandsOnDAY26\bin\Debug\net8.
0\HandsOnDAY26.exe (process 19044) exited with code 0 (0x0).
To automatically close the console when debugging stops, enable Tools->Options->Debugging
->Automatically close the console when debugging stops.
Press any key to close this window . . .|
```